The Drink Exchange — Production Build Brief

> **Use this as the master project prompt** for Claude Code, Cursor, v0, Bolt, or any other AI coding tool. Paste it as a project brief, then drive build phases from it. Treat each Phase below as a separate coding session.

---

0. Context (read this first)

You are building the live pricing, point-of-sale, and customer-display system for **The Drink Exchange**, a real cocktail bar opening in Adelaide, South Australia. The concept is a market-driven bar: drink prices float in real time based on demand, exactly the way a stock exchange works. Customers can see live prices on screens around the venue. The operator can trigger "market crashes" that discount every drink simultaneously for a short window.

This is not a hackathon project or a demo. **Real money, real drinks, real customers.** Code must be production-grade: persistent state, real-time sync across multiple clients, graceful degradation, full auditability of every transaction. A POS that loses an order or shows the wrong price on a Saturday night is a failure of the business, not just the software.

A working single-page prototype already exists (React + Recharts + Tailwind, in-memory state, no auth, no persistence). Use it as a reference for the pricing math and UI direction, but assume nothing else about the architecture. The production system is a different beast.

Operator profile

Solo developer building this as a working bar concept. Comfortable with:
- Node.js, TypeScript, React, Next.js
- Shopify, Railway hosting, Postgres
- iOS/Swift (separate concern; not in scope here)
- Tailwind, shadcn/ui

Preferences:
- **No em dashes anywhere** in code comments, UI copy, or documentation. Use regular hyphens, commas, or restructure.
- Casual, direct code style. Comments where they add value, not as decoration.
- Prefer boring tech that ships over clever tech that breaks.

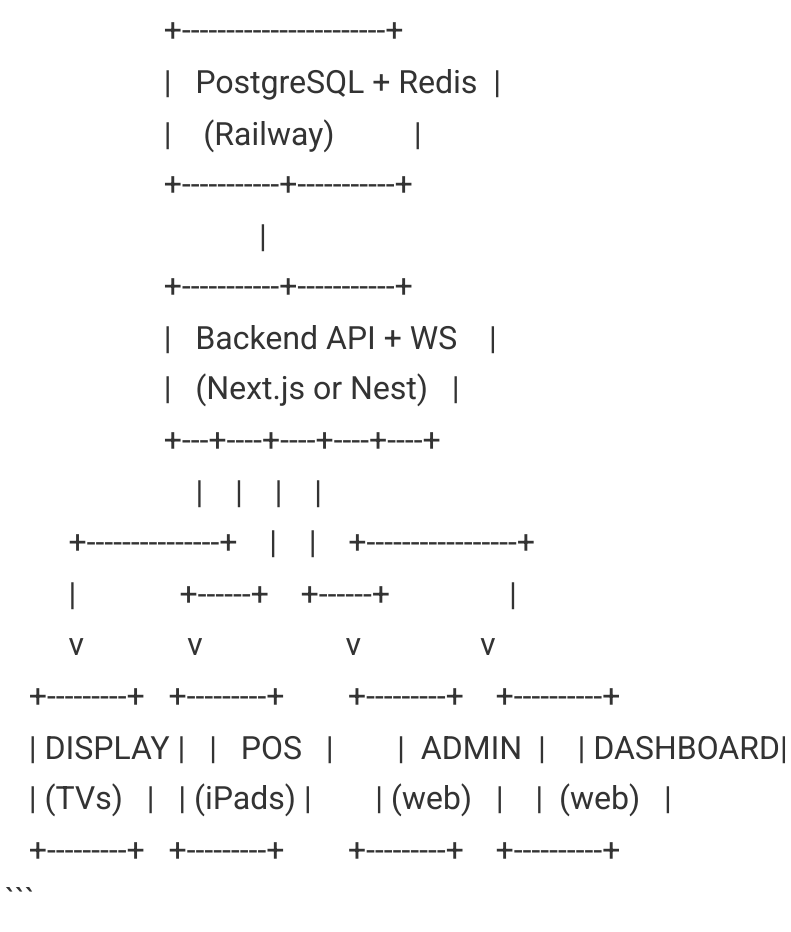
Target deployment

- **Hosting:** Railway (Postgres + web service)
- **Domain:** thedrinkexchange.com.au (or similar)
- **Currency:** AUD, GST-inclusive pricing (10% GST baked into displayed price)
- **Timezone:** Australia/Adelaide (ACST/ACDT, GMT+9:30 / +10:30)
- **Locale:** en-AU

---

1. System Overview

Four logical clients, one backend, one source of truth.



Display — read-only, fullscreen on smart TVs around the venue. No interaction. Pure ticker + drink cards + crash banner. Multiple screens render the same view (or rotated panels of it).

POS — staff-facing tablets behind the bar. Touch-optimised. Place orders, process payment, void/refund. Auth required.

Admin — web-based, used by ownership and venue manager. Trigger crashes, edit menu, adjust market parameters, manage staff, view shift reports. Strong auth required.

Dashboard — web-based, read-mostly. Live analytics for ownership: revenue, sales velocity, top movers, recent orders.

All four clients connect to the same backend and the same realtime channel.

---

2. Tech Stack (recommended, not mandatory)

Pick the smallest stack that delivers reliability. Suggested defaults:

Layer	Choice	Why
Framework	**Next.js 15 (App Router)**	Single deploy unit on Railway. API routes, RSC for dashboard, client components for POS/display.
Database	**PostgreSQL** on Railway	Persistent, transactional, fine for this scale (well under 100k orders/year initial).
ORM	**Drizzle** or Prisma	Drizzle if you want speed and SQL closeness; Prisma if you want introspection. Either works.
Realtime	**Pusher Channels** or **Supabase Realtime**	Both Railway-compatible. Pusher is dead simple to integrate. Avoid rolling your own Socket.io unless you have a strong reason.
Auth	**Better Auth** or Clerk	Better Auth if self-hosted preferred; Clerk if speed matters more than cost.
UI	**Tailwind + shadcn/ui**	Match the artifact prototype aesthetic.
Charts	**Recharts**	Already used in the prototype.
Payments	**Stripe Terminal** (primary), Square (alternative)	Adapter pattern so provider can be swapped.
Cache / pub-sub locally	**Redis** (Railway add-on)	Price tick state, rate limiting, session cache.
Background jobs	**Inngest** or BullMQ	End-of-day Z-reports, scheduled crashes, email/SMS receipts.
Logging	**Axiom** or Better Stack	Anything that ingests JSON logs with retention.
Error tracking	**Sentry**	Standard.

If the AI tool building this prefers a slightly different stack, that's fine, but **do not** introduce more than one experimental dependency. This system must run for years.

---

3. The Pricing Engine (spec — do not freelance the math)

Every drink has the following attributes:

```typescript
interface Drink {
  id: string;
  name: string;
  category: 'Cocktails' | 'Beer' | 'Wine' | 'Spirits' | 'Shots' | 'Non-Alc';
  emoji: string;                  // displayed on POS and customer screens
  basePrice: number;              // AUD inc. GST, the "fair value"
  currentPrice: number;           // AUD inc. GST, what's displayed RIGHT NOW
  costPrice: number;              // pour cost, used for margin floor enforcement
  minPriceMultiplier: number;     // default 0.5 (cannot drop below 50% of base)
  maxPriceMultiplier: number;     // default 2.5 (cannot rise above 250% of base)
  isDynamic: boolean;             // false = fixed-price (house beer, etc.); true = floats
  isActive: boolean;              // is it on the menu right now
  sortOrder: number;
  createdAt: Date;
  updatedAt: Date;
}
```

3.1 Price tick (mean reversion)

A scheduled job runs **every 2 seconds** (per-second feels jumpy; per-5 feels dead).

For each dynamic drink:

```
drift   = (basePrice - currentPrice) * decayRate
noise   = (random in [-1, +1]) * basePrice * noiseLevel
newPrice = clamp(
           currentPrice + drift + noise,
           basePrice * minPriceMultiplier,
           basePrice * maxPriceMultiplier
         )
```

Defaults: `decayRate = 0.04`, `noiseLevel = 0.004`.

The result is broadcast on the realtime channel as a `price.tick` event. Every tick is also persisted to a `price_history` table so the dashboard can chart it.

3.2 Order impact

When an order is placed (POS → API), each line item bumps the price of its drink multiplicatively:

```
impact   = (1 + volatility) ^ quantity
newPrice = min(currentPrice * impact, basePrice * maxPriceMultiplier)
```

Default `volatility = 0.05`. Ordering 4 of the same drink at once moves it `(1.05)^4 ≈ 1.216x`.

Order impact is applied **before** the next scheduled tick, and an immediate `price.update` is broadcast so the screens visibly react to the purchase.

3.3 Crash mechanic

A crash is **a temporary display-and-charge discount** applied uniformly across all dynamic drinks. It does **not** modify `currentPrice` in the database. Instead, an active crash event exists:

```typescript
interface CrashEvent {
  id: string;
  startedAt: Date;
  endsAt: Date;
  discountPercent: number;        // e.g. 0.30 for 30% off
  triggeredBy: string;            // user ID
  triggeredVia: 'manual' | 'scheduled' | 'social' | 'event';
  totalOrdersDuringCrash: number;  // updated as orders come in
  totalRevenueDuringCrash: number;
  cancelledEarly: boolean;
}
```

When a crash is active:
- Display price for every dynamic drink is `currentPrice * (1 - discountPercent)`.
- POS automatically applies the crash discount to the order total. Staff cannot accidentally charge a non-crash price.
- Margin floor enforced: if `displayPrice < costPrice * (1 + minMarginMultiplier)` (default 0.30, i.e. 30% margin floor), the displayed price is clamped at the floor. The system logs which drinks hit the floor for the operator.
- When the crash ends (timer expires or operator ends it manually), the screens revert. **`currentPrice` is not reset.** The market continues from wherever it was.

3.4 Live-tape generation

In addition to the price tick, the backend maintains a rolling **24-hour price history** per drink, sampled every 2 seconds, capped at the last 2,000 points (more than enough for a trading day). This is the data source for:
- Sparklines on customer display cards
- The full chart in the dashboard
- The "biggest mover today" and "best value right now" automatic content posts

3.5 Out-of-trading-hours behaviour

When the bar is closed, the price engine pauses. Last-known prices are frozen. When the bar opens for the next trading session, prices reset to base, except for drinks that have been manually carried over (admin can flag specific drinks to retain overnight price for narrative continuity).

---

4. Data Model

```typescript
// Core domain
Drink             // see Section 3
DrinkCategory     // optional table if categories become complex
Order             // header
OrderLine         // each drink ordered
OrderPayment      // 1:1 with order; may have multiple if split
PriceHistory      // every tick, per drink
CrashEvent        // see Section 3.3

// Operational
User              // staff and admin accounts
Role              // 'staff' | 'manager' | 'admin' | 'owner'
Shift             // open/close session; ties orders to a trading period
ShiftReport       // end-of-day reconciliation (Z-report)
Setting           // KV store for volatility, decayRate, noiseLevel, etc.

// Audit
AuditLog          // who did what when. Crashes, refunds, price overrides, etc.
```

Order schema

```typescript
interface Order {
  id: string;               // ULID
  orderNumber: number;      // human-readable, per-shift
  shiftId: string;
  staffId: string;
  status: 'open' | 'paid' | 'voided' | 'refunded';
  subtotal: number;         // sum of lines
  gstAmount: number;        // already included in displayed prices, but stored separately for reports
  total: number;
  crashEventId: string | null;  // populated if order was placed during a crash
  crashDiscount: number;    // 0 if no crash, otherwise the percent off
  paymentMethod: 'card' | 'cash' | 'split';
  paidAt: Date | null;
  voidedAt: Date | null;
  voidReason: string | null;
  refundedAt: Date | null;
  refundAmount: number;
  notes: string | null;
  createdAt: Date;
}

interface OrderLine {
  id: string;
  orderId: string;
  drinkId: string;
  drinkNameSnapshot: string;    // immutable copy of name at time of sale
  basePriceSnapshot: number;    // immutable
  pricePaid: number;            // per unit, after crash discount
  marketPriceAtPurchase: number; // pre-crash price (for analytics)
  quantity: number;
  lineTotal: number;
  createdAt: Date;
}
```

Every numeric column is `numeric(10, 2)` for currency. No floats.

---

5. Real-Time Sync

The realtime channel is the spine of the system. Every client subscribes; every state change broadcasts.

5.1 Channels

- `prices` — public read; price ticks and post-order updates
- `crash` — public read; crash start/end/tick countdown
- `orders` — authenticated; order events for the dashboard
- `admin` — admin-only; menu changes, settings changes

5.2 Events

Event	Payload	Triggered by	Consumers
`price.tick`	`{drinkId, currentPrice, timestamp}[]`	Scheduled job every 2s	Display, Dashboard
`price.update`	same shape, single drink usually	Order placed, admin override	Display, POS, Dashboard
`crash.started`	`CrashEvent`	Admin action, scheduled, or event trigger	All clients
`crash.tick`	`{remainingSeconds}`	Server-side timer, every 1s	All clients
`crash.ended`	`{crashEventId, cancelledEarly}`	Timer expiry or admin action	All clients
`order.placed`	`Order` (sanitised)	Order creation	Dashboard, POS (the originating one acknowledges, others ignore)
`drink.updated`	`Drink`	Admin edit	All clients
`settings.updated`	`Setting[]`	Admin edit	Engine (re-reads on next tick)

5.3 Reconnection

Every client must:
1. On connect, fetch a snapshot (`GET /api/state/snapshot`) before subscribing to events.
2. Apply events incrementally from the snapshot point.
3. If the connection drops for more than 30 seconds, refetch the snapshot rather than replaying missed events.

---

6. The Display Client (Customer-Facing Screens)

This is the **iconic** part of the system. The brand lives or dies on what's on these screens.

6.1 Layout

- Fullscreen, 16:9 or 9:16 depending on screen orientation

- No browser chrome (smart TVs run Chrome in kiosk mode)
- Auto-reconnects on network loss with visible "RECONNECTING…" state
- No clicks, no interactions, no cursor

6.2 Three rendering modes

1. **Default — Market Open.** Header with bar logo + "MARKET OPEN" + clock. Scrolling tape across the top. Grid of drink cards. Each card: emoji, name, category, current price, % change vs base, sparkline of last 25 ticks. Live update.

2. **Crash Mode.** Full-screen red gradient takeover for the first 3 seconds (with audio cue, see 6.4). After 3s, returns to grid layout but with: every card showing strikethrough original price + bold discounted price, persistent red banner with countdown, ticker tape in red text. Stays in this mode until crash ends.

3. **Closed.** Outside trading hours. Last-known prices shown frozen. "MARKET CLOSED" indicator. Opening time displayed.

6.3 Multiple screens

Three rendering profiles, selectable per screen via a URL param (`/display?profile=main`):

- `main` — full grid (best for the largest screen, behind the bar)
- `tape` — full-screen scrolling tape only (best for narrow displays beside the entrance)
- `featured` — three featured drinks at maximum size, rotates every 30s (best for tabletop screens)

All profiles subscribe to the same backend, render the same source of truth.

6.4 Audio

The display client emits an audio cue when a crash starts. A short stinger (1.5s, like a stock-market opening bell). Modern browsers require a one-time user interaction before audio can play. Solution: on screen first-load, show a "TAP TO ENABLE SOUND" overlay; once tapped, the screen never needs to be interacted with again unless reloaded.

6.5 Failure handling

If the backend is unreachable:
- Show last-known prices, with a small "OFFLINE" indicator
- Do not show stale price moves; freeze the sparklines
- Reconnect aggressively; resume immediately on reconnection

---

7. The POS Client (Staff Tablets)

Touch-first. Built for fast service in a noisy bar at midnight on Saturday by someone who has been on their feet for six hours.

7.1 Hardware target

- Apple iPad 10th gen or newer, 10.9", landscape
- Network: WiFi primary, 4G/5G fallback via a dual-WAN router (do not assume always-online)
- Card reader: Stripe Terminal BBPOS WisePOS E (primary) or Square Reader

7.2 Core flow

1. Staff logs in with PIN (no full passwords on tablets)
2. Selects "Open Order" or scans an open order tab
3. Taps drinks to add to cart
4. Cart shows live prices that update if the market moves
5. Customer confirms order; staff taps "CHARGE"
6. Card terminal handles payment
7. Order completes; price impact applied to market
8. Digital receipt (SMS or email) optionally sent
9. Tab can be left open for additional rounds

7.3 Cart behaviour

- Prices in the cart are **locked at the moment the customer agrees to charge**, not when added to the cart. If a price changes between adding and charging, the cart shows the change visibly so staff can decide whether to refresh or proceed.
- If a crash starts while there's an open cart, prices in the cart automatically update to the crash price and the staff sees a "CRASH ACTIVE" banner on the cart.
- Voids: staff can void any line before charging, no auth required. After charging, void requires manager PIN.

7.4 Offline tolerance

- The POS caches the last-known menu + prices in localStorage / IndexedDB.
- If network drops mid-shift, the POS continues to take orders, queuing them locally. Prices freeze at last-known.
- When network returns, queued orders are sent to the backend in order. Conflicts (e.g. a drink was deleted while offline) are flagged for manager review.

- A small banner indicates "OFFLINE MODE — orders queued: 3" so staff are aware.

7.5 End-of-shift

The senior staff member closes the shift via POS. The system:
1. Stops accepting new orders
2. Generates a Z-report: total revenue, GST, COGS estimate, drink counts, crash events summary, voids and refunds
3. Emails the Z-report to ownership
4. Persists the shift summary

---

8. The Admin Client (Operator Controls)

Web-based, desktop-first but mobile-responsive. Strong auth (TOTP 2FA required).

8.1 Pages

Menu — CRUD for drinks. Bulk price adjustments. Activate/deactivate drinks without deleting (preserves historical reporting integrity). Drag-to-reorder within categories. Image upload to S3 or equivalent.

Market Controls — Live preview of current prices. Sliders for `volatility`, `decayRate`, `noiseLevel`, `minMultiplier`, `maxMultiplier`. Changes apply on the next tick. An "Override" button lets the operator manually set the current price of any drink (logged in audit log).

Crash Centre — The most important page in the system.
- Big "Trigger Crash Now" button (red, confirmation modal: "This will discount every dynamic drink by X% for Y minutes. Proceed?")
- Crash schedule: list of scheduled crashes (e.g. "Friday 21:00 — 25% off — 3 minutes")
- Crash history: every crash that's ever happened, with sales impact metrics
- Social trigger config: webhook URL that, when called, triggers a crash. Used for "comment CRASH on this post and we'll crash the market" campaigns. Rate-limited.

Staff — User management. Add staff, assign roles, deactivate. Set PIN. View per-staff sales performance.

Shifts — Past shift reports. Re-export Z-reports. Reconcile cash discrepancies.

Settings — Trading hours per day, GST rate (10% default, locked), receipt template, branding, payment provider config.

Audit Log — Every important action: crashes, price overrides, refunds, staff role changes, menu changes. Searchable and filterable. Cannot be deleted.

8.2 Permission matrix

Action	Staff	Manager	Admin	Owner
Place order	✓	✓	✓	✓
Void line (pre-charge)	✓	✓	✓	✓
Void order (post-charge)		✓	✓	✓
Refund		✓	✓	✓
Trigger crash		✓	✓	✓
Edit menu / prices			✓	✓
Manage staff			✓	✓
View financials		summary	✓	✓
Change market parameters			✓	✓
Manage payment config				✓

---

9. The Dashboard (Live Analytics)

Web-based, read-mostly. Designed to live on a manager's phone or a back-office screen.

9.1 KPIs

- Revenue today (live)
- Drinks sold today (live)
- Average order value
- Market index (average % change across the menu)
- Active crash status

9.2 Charts

- Revenue per hour (today, with yesterday and last-week-same-day overlays)
- Top 4 movers (% change line chart, last 60 minutes)
- Sales velocity (drinks per 10-minute window)

9.3 Feeds

- Live order feed (most recent 20, auto-scrolling)

- Crash events feed (today)
- Margin alerts (any drink that's been at floor for >5 minutes)

9.4 Daily summary

Cron job at 03:00 ACDT emails a daily summary to ownership: yesterday's revenue, biggest mover, biggest seller, crashes, anomalies. Plain HTML email, no marketing fluff.

---

10. Brand & Visual Direction

The brand is defined; the build must respect it.

10.1 Identity

- **Name:** The Drink Exchange
- **Tagline:** Trade Drinks. Not Stocks.
- **Logo mark:** wordmark "THE DRINK EXCHANGE" with the I in DRINK replaced by a green/red candlestick chart icon
- **Voice:** confident, witty, never corporate. The brand sounds like a financial newsletter written by someone who'd rather be at a bar.

10.2 Colours

```
Primary green (bull):   #10B981
Crash red (bear):       #EF4444
Background dark:        #09090B / #18181B
Card dark:              #18181B / #27272A
Text primary:           #FAFAFA
Text secondary:         #A1A1AA
Border:                 #27272A
Highlight amber:        #FBBF24  (used sparingly)
```

10.3 Typography

- **Display / numbers:** any clean monospaced font with tabular figures. JetBrains Mono, IBM Plex Mono, or Geist Mono. Prices and percentages MUST be monospaced for stable column alignment.
- **UI / body:** Inter, Geist, or system sans-serif.

- **Headlines on social and signage:** Bold condensed sans (Bebas Neue or similar).

10.4 Motion

- Price changes animate over 400ms, easing-out
- Crash entrance: full-screen red flash 0% to 80% opacity over 200ms, then settle
- Ticker tape: linear, infinite scroll at ~120px per second
- Sparklines redraw without animation (the data IS the animation)

10.5 Audio

- Crash stinger: 1.5s, low-to-high sweep with a bell tail
- Order ding (POS only): 200ms soft chime on successful charge
- All audio respects a global mute toggle in admin settings

---

11. Payments, Receipts, Compliance

11.1 Payment flow

Adapter pattern. The system talks to a `PaymentProvider` interface; concrete implementations for Stripe Terminal and Square. Adding a new provider should require only a new adapter class.

Required operations per provider:
- `chargeAmount(orderId, amountCents, terminalId): Promise<PaymentResult>`
- `refundCharge(chargeId, amountCents): Promise<RefundResult>`
- `getTerminalStatus(terminalId): Promise<'connected' | 'disconnected' | 'busy'>`

11.2 Receipts

- Default to digital: SMS via Twilio or email via Postmark/Resend
- Printed receipts optional: support Epson TM-m30 over network printing (ESC/POS); leave behind an interface to add it in Phase 5

11.3 GST and reporting

- All displayed prices are GST-inclusive (Australian convention)
- Z-reports show ex-GST subtotal, GST, total
- BAS quarter export: CSV of all orders with GST broken out, downloadable from admin

11.4 Liquor licensing notes

The system must support:
- Per-shift max-discount cap (configurable; if a crash would breach it, the crash is auto-throttled)
- Cooling-off window enforcement: minimum X minutes between consecutive crashes (default 20 minutes)
- Trading hours hard-stop: no orders accepted outside licensed trading hours

These are configurable in admin and exist to keep the operator compliant with the Liquor Licensing Act 1997 (SA).

---

12. Build Phases

Phased build. Each phase ships independently and is deployable.

Phase 1 — The Ticker (week 1-2)

Goal: a live, beautiful, single customer display running off seed data, with no POS yet.

- Next.js project skeleton, Tailwind, shadcn/ui
- Postgres schema for drinks and price history
- Drizzle migrations
- Seed data: the 14 starter drinks from the prototype
- Price tick engine running on a server-side scheduler
- Display client at `/display?profile=main`
- Pusher integration
- Brand-correct visuals (logo, colours, ticker tape, sparklines)
- Deployed to Railway behind a real domain

Acceptance: the operator opens `display.thedrinkexchange.com.au` on a TV, sees prices drifting in real time, sparklines updating. Closes the browser and re-opens; prices resume from where they were. The single most important visual asset of the bar exists and is beautiful.

Phase 2 — Manual Crashes (week 3)

Goal: the operator can trigger crashes manually from a basic admin page. No auth yet, just a hidden URL.

- Admin page at `/admin/crash` with a Trigger Crash button
- Crash event persistence and broadcasting
- Display client renders crash mode correctly

- Audio cue
- Margin-floor enforcement

Acceptance: operator hits the button; every screen erupts; countdown runs; reverts cleanly.

Phase 3 — POS (week 4-5)

Goal: orders can be placed, payment can be taken, prices move.

- Auth (Better Auth or Clerk), PIN-based for staff
- POS client at `/pos` optimised for iPad
- Cart, order placement, order-impact engine
- Stripe Terminal integration
- Digital receipts
- Order persistence
- Offline mode (basic: queue and replay)

Acceptance: real card payment placed on a Stripe test terminal; price visibly moves on the display screen; receipt arrives by email; order appears in the database.

Phase 4 — Admin and Dashboard (week 6-7)

Goal: the operator can run the bar without touching a database.

- Full admin UI (menu, market controls, crash centre, staff, settings, audit log)
- Live dashboard
- Shift management and Z-reports
- Daily email summary
- 2FA on admin accounts
- Audit logging everywhere

Acceptance: operator can onboard a new staff member, edit the menu, schedule a crash for Friday at 9pm, view yesterday's report, and lock down the system, without engineering help.

Phase 5 — Hardening (week 8+)

Goal: production-ready.

- Stress test: simulate 200 concurrent orders in 5 minutes
- Sentry, Axiom, uptime monitoring
- Backups: nightly Postgres dump to S3-compatible storage

- Disaster recovery runbook
- Receipt printer integration
- Social crash webhook
- BAS export
- Documentation: operator manual, staff training cheat sheet

Acceptance: all of Phase 1-4 still works, plus the system survives a simulated bad night.

---

13. Non-Negotiables (do not skip)

1. **No floats for money.** Use `numeric(10, 2)` in Postgres, integer cents in code (`amountCents: number`), or a decimal library. Floats lose pennies and the auditor will find them.
2. **Every state-changing action is logged** in the audit log with timestamp, actor, and before/after value.
3. **POS works offline.** Internet outages are not an excuse to lose orders.
4. **Crash floor is enforced server-side.** A malicious client cannot bypass it.
5. **Snapshot-then-subscribe** for realtime. Never rely solely on event replay.
6. **All timestamps stored in UTC**, rendered in Australia/Adelaide.
7. **Receipts are immutable.** Once paid, an order's line items and prices cannot be edited; only the order can be refunded.
8. **The display never crashes.** If the backend is unreachable, it shows last-known prices with an "OFFLINE" indicator. It does not show a JavaScript error.
9. **2-second tick rate is sacred.** Slower and the market feels dead; faster and it feels jittery. Do not change without testing on real hardware.
10. **No em dashes** in any UI copy or code comments. Standard hyphen, comma, or restructure.

---

14. Out of Scope

For the initial build, the following are explicitly **not** required and should not consume time:

- Native mobile apps (POS is a PWA on iPad)
- Customer-facing ordering or pre-order
- Table reservations (use an existing tool like Now Book It or SevenRooms)
- Loyalty programme (Phase 6 candidate)
- Multi-venue / franchise support (single venue only for now)
- Inventory cost tracking and pour reconciliation (manual via existing tools)
- Anything involving the "stock market" theme beyond what's specified (no portfolios, no virtual currencies, no trading competitions)

If a feature is tempting and not in this brief, write a one-line note in `BACKLOG.md` and move on.

---

15. Useful Reference

The original single-page prototype lives at `BarStockExchange.jsx` (provided separately). It demonstrates:
- The pricing math (Sections 3.1 and 3.2 are derived from it)
- The visual aesthetic for the display
- The POS cart UX

Treat it as a wireframe and a visual reference. Do not port its in-memory state model; build it properly for production.

---

16. Acceptance: definition of done for the whole system

The bar opens. On a Friday night, the operator:

1. Arrives at 16:00, opens admin, opens the bar (starts the shift).
2. Walks past four screens around the venue, all showing the live market.
3. Watches the first customer place a $14.80 Espresso Martini order on the POS; sees the price tick up on the screen near the entrance within a second.
4. At 21:00 sharp, the scheduled crash fires; every screen erupts; customers rush the bar; the POS handles 22 orders in 90 seconds without dropping one.
5. Closes the shift at 02:00; Z-report emails to ownership; daily summary lands at 03:00.
6. Looks at the dashboard on Saturday morning, sees the night's narrative in chart form, and adjusts the volatility setting up by 0.005 for tonight.

That's the system. Build it well.